

Bridging the Semantic Gap in Process Mining

A Deterministic, WebAssembly-Accelerated Architecture for Formal Pipeline Verification

Sean Chatman

*A dissertation submitted in partial fulfillment
of the requirements for the degree of*
Doctor of Philosophy

Department of Computer Science

June 2026

Abstract

Process mining sits at the intersection of data science and business process management, heavily reliant on high-level, dynamically typed languages like Python (e.g., PM4Py) for rapid prototyping and algorithmic discovery. However, as process mining matures into a production-critical operational technology, this reliance introduces significant epistemological and engineering vulnerabilities: non-deterministic execution environments, high computational overhead, and a lack of formal verifiability across pipeline boundaries.

This thesis presents a novel, dual-layered architecture designed to resolve these limitations. First, we introduce **wasm4pm**, a highly optimized, deterministic computational substrate written in Rust and compiled to WebAssembly (WASM). This substrate provides mathematically rigid guarantees for event log parsing, algorithm execution, and typestate management. Second, we introduce a semantic bridge utilizing the Language Server Protocol (LSP). By developing **tower-lsp-max**—a highly concurrent, asynchronous Rust framework for LSP implementations—and its domain-specific consumer, **pm4py-lsp**, we bring formal static analysis, real-time diagnostic evaluation, and typestate tracking directly into the data scientist’s localized Python environment.

Through the rigorous application of the “Ostar Proof Discipline,” we demonstrate how continuous, combinatorial parity can be maintained between the volatile Python frontend and the deterministic WASM backend, ensuring that every analytical claim is backed by a cryptographically verifiable execution receipt.

Acknowledgments

This work is the culmination of years of iterative engineering, architectural refactoring, and theoretical exploration. I would like to thank the open-source communities surrounding the Rust programming language, WebAssembly, and PM4Py. Building `tower-lsp-max` would not have been possible without the foundational work of the `tower-lsp` maintainers. To the various engineers, data scientists, and agents who have engaged with, critiqued, and dismantled early iterations of the Ostar Generative Pipeline: your relentless pursuit of combinatorial maximalism drove this research to its absolute limit.

Contents

1	Introduction	1
1.1	The State of Process Mining	1
1.2	The Hypothesis of Combinatorial Maximalism	1
2	The Computational Substrate: wasm4pm	3
2.1	Architectural Foundations	3
2.1.1	Memory Management and Event Log Topologies	3
2.1.2	Object-Centric Process Queries (OCPQ)	3
3	Semantic Bridging via the Language Server Protocol	5
3.1	The Limits of Static Analysis	5
3.2	Developing <code>tower-lsp-max</code>	5
3.3	The <code>pm4py-lsp</code> Implementation	5
4	The Ostar Proof Discipline	7
4.1	Receipts and Cryptographic Homeomorphism	7
4.2	Closing the Taxonomy Gap	7
5	Conclusion	9
5.1	Summary of Contributions	9
5.2	Future Work	9

List of Figures

Chapter 1

Introduction

1.1 The State of Process Mining

Over the past decade, process mining has evolved from a niche academic discipline into a foundational technology for enterprise operational transparency. Frameworks such as PM4Py have democratized access to complex discovery algorithms (e.g., the Inductive Miner, Alpha Miner) and conformance checking utilities. Python’s dominance in this sphere is unquestioned, driven by its unparalleled ecosystem for data manipulation (pandas, polars) and machine learning.

However, this accessibility comes at a severe structural cost. Python is dynamically typed and executed via an interpreter constrained by the Global Interpreter Lock (GIL). When applied to enterprise-scale event logs—often containing tens of millions of discrete events across complex, concurrent, object-centric (OCEL) relational graphs—the standard toolchain degrades.

More critically, the lack of formal, compile-time guarantees introduces what we term an “epistemological crisis” in process intelligence. If a process mining pipeline cannot guarantee deterministic execution across varying environments, or if an analyst can inadvertently execute discovery algorithms on unformatted, malformed dataframes without immediate structural feedback, the resulting analytical claims are fundamentally suspect.

1.2 The Hypothesis of Combinatorial Maximalism

This thesis proposes that the solution is not to abandon the Python ecosystem, but to structurally bound it. We hypothesize a paradigm of *Combinatorial Maximalism*: the rigorous, exhaustive verification of all pipeline states through formal contracts, backed by a high-performance, deterministic execution substrate.

We assert that by mapping the Abstract Syntax Tree (AST) of the data scientist’s high-level workflow via the Language Server Protocol (LSP), and binding those semantic tokens to isomorphic functions in a strictly typed WebAssembly kernel, we can achieve the rapid prototyping benefits of Python with the mission-critical safety and performance of Rust.

Chapter 2

The Computational Substrate: wasm4pm

2.1 Architectural Foundations

To provide deterministic parity, we engineered **wasm4pm**. Written in Rust, it is designed from the ground up to bypass the limitations of traditional garbage-collected runtimes.

2.1.1 Memory Management and Event Log Topologies

Standard event log parsing involves significant string allocation overhead. **wasm4pm** utilizes a columnar, zero-copy architecture for parsing XES and OCEL formats. By leveraging memory-mapped I/O and strictly avoiding unnecessary allocations, the parsing subsystem operates at the absolute bounds of the host’s disk read speed.

2.1.2 Object-Centric Process Queries (OCPQ)

The shift from traditional, single-case-identifier event logs to Object-Centric Event Logs (OCEL 2.0) introduces immense relational complexity. Events are no longer isolated to a single trace but exist as hyper-edges in a bipartite graph of objects and activities. **wasm4pm** implements a dedicated **ocpq** engine that models these relationships explicitly, allowing for constant-time traversal of causal dependencies, which is critical for real-time conformance checking.

Chapter 3

Semantic Bridging via the Language Server Protocol

3.1 The Limits of Static Analysis

Traditional linters and static analyzers evaluate code for syntactic correctness or stylistic adherence. They do not understand domain-specific semantics. In process mining, executing a Directed Follows Graph (DFG) discovery on a dataframe that lacks standardized `case_id`, `activity_key`, and `timestamp_key` mappings will either crash at runtime or silently produce garbage data.

3.2 Developing tower-lsp-max

To bridge this gap, we required an LSP server capable of deep semantic inference. Existing Rust LSP frameworks were either overly monolithic or lacked the asynchronous concurrency models required for analyzing massive data environments in real-time.

We developed `tower-lsp-max`, expanding upon the standard `tower-lsp` framework. By heavily leveraging the `tower` ecosystem’s middleware patterns, `tower-lsp-max` allows for robust, composable language servers. It fully supports the LSP 3.18 specification, handling requests via asynchronous streams, ensuring that background parsing of multi-gigabyte `.xes` files does not block the user’s UI thread.

3.3 The pm4py-lsp Implementation

Using `tower-lsp-max` as our foundation, we authored `pm4py-lsp`. This language server operates entirely within the data scientist’s IDE. It tracks the typestate of variables.

When a user writes:

```
1 import pandas as pd
2 import pm4py
3
4 df = pd.read_csv("hospital_billing.csv")
5 net, im, fm = pm4py.discover_petri_net_inductive(df)
```

The `pm4py-lsp` evaluates the AST. It recognizes that `df` is a raw, unformatted dataframe. It immediately raises a diagnostic, preventing the erroneous execution. Furthermore, it provides a *Code Action* (a quick fix) that mutates the document to safely map the dataframe prior to discovery.

Beyond simple diagnostics, the LSP implements `textDocument/hover` and `textDocument/codeLens`. Hovering over the `discover_petri_net_inductive` invocation reveals its formal mathematical properties and its direct execution mapping within the `wasm4pm` kernel.

Chapter 4

The Ostar Proof Discipline

4.1 Receipts and Cryptographic Homeomorphism

A central tenet of this thesis is the "One-Line Law: No receipt, no claim." It is insufficient to merely execute code; the execution must be proven.

For every algorithmic operation $\mathcal{O}$ in `wasm4pm`, the state mutation is hashed using BLAKE3. This produces a deterministic, cryptographically secure receipt. If an automated agent or a human analyst claims a workflow is valid, they must produce the corresponding `receipt.json`.

4.2 Closing the Taxonomy Gap

During the development lifecycle, we identified a "Taxonomy Gap" (Checkpoint PM4PY-LSP-002) where local test fixtures were successfully evaluating parity, but failing to persist physical receipts to the global taxonomy for release verification. By enforcing strict file-system persistence during testing and integrating these artifacts into the global `release:full` pipeline, we closed the gap, achieving the PM4PY-LSP-004_ALIVE state. Every algorithm in the suite (60/60) now possesses mathematically sound, persistent reachability and behavior evidence.

Chapter 5

Conclusion

5.1 Summary of Contributions

This research demonstrates that the flexibility of dynamic data science environments need not come at the expense of formal engineering rigor. By treating the Language Server Protocol not merely as a tool for autocompletion, but as a continuous, topological bridge between high-level intent (Python/PM4Py) and low-level deterministic execution (Rust/WASM), we have established a new paradigm for process intelligence.

5.2 Future Work

While `wasm4pm` and `tower-lsp-max` provide a robust foundation, several avenues remain open. The automatic compilation of Python ASTs directly into WASM bytecode via the LSP interface remains an ongoing engineering challenge. Furthermore, expanding the `ocpq` engine to support distributed, multi-node clustering for planetary-scale event logs will be the focus of subsequent research.